Absorción refinada: complemento de formalización y reproducibilidad

Juan Pablo Traverso Gianini
Investigador independiente, Santiago, Chile
jtraverso@gmail.com
ORCID: 0009-0003-6068-4096

Complemento de *Formalización de la absorción refinada: interfaces de prueba y consecuencias*, research note, prepublicación 1.0.

Borrador editorial — 27 de octubre de 2026. El freeze formal está completo; quedan pendientes la reserva del DOI y la publicación de los documentos.

© 2026 Juan Pablo Traverso Gianini. Licencia CC BY 4.0.

Resumen

Este complemento identifica las declaraciones demostradas, las versiones de las fuentes y la evidencia de reproducibilidad que acompañan la research note. El freeze formal contiene 316 fuentes locales de Lean y la evidencia de una campaña de Lake con salida del proyecto inicialmente vacía, reutilizando cachés de dependencias fijadas. Sus 360 consultas de axiomas usan sólo los tres axiomas fundacionales indicados, o ninguno. Distinguimos la compilación local completa, la raíz agregada de imports, las interfaces seleccionadas de comparación y la verificación separada de Palomar. También registramos la interfaz añadida para vértices ambientes, la precisión sobre encajes de la survey, el alcance exacto de intersecciones y calendarios, y las pruebas externas fijadas por commit que cierran el ensamblaje cuantitativo.

1. Fuentes matemáticas y alcance

Los dos textos principales son Delcourt–Postle [1], con base de enunciados arXiv:2402.17855v1, y Postle [2], desarrollado contra arXiv:2510.19978v1. El identificador publicado de la survey es DOI 10.1137/25M1805977, ICM 2026, volumen 6, páginas 212–231. Su Conjetura 1.1, Teorema 1.10, Definiciones 3.6–3.7 y Lema 3.8 se comprobaron en esa edición. Es una comprobación de los enunciados identificados, no de equivalencia entre todos los contenidos de ambas ediciones.

La formalización contiene los resultados principales de absorción, descomposición densa y existencia de diseños que se enumeran abajo. El resultado de índice arbitrario demuestra la existencia de familias simples de bloques para todos los $q > r \geq 2$ y $\lambda \geq 1$ fijos, en todos los órdenes admisibles suficientemente grandes. No está restringido al caso de Steiner. Las aplicaciones generales de cintura alta quedan fuera de este alcance.

Los autores de los teoremas fuente conservan la atribución matemática. La nota explica interfaces y deduce consecuencias de resultados demostrados. Se reconoce el trabajo clásico sobre intersecciones de sistemas triples [10,11]; no se reivindica prioridad histórica para cada consecuencia de parámetros generales. El resultado de Nash–Williams se atribuye a Delcourt–Postle [7], y la prueba formal reutilizada está documentada en [8,9].

2. Resultados canónicos

Las tablas siguientes identifican las dieciocho declaraciones seleccionadas por las tres configuraciones de comparación. Sus tipos completos están en los Challenges independientes; los intervalos de extracción y hashes están en `provenance/palomar-extractions.json`. Cada nombre seleccionado aparece en las salidas congeladas de consultas de axiomas. La biblioteca contiene otras declaraciones demostradas, además de esta selección.

2.1. Artículo original

Resultado fuente	Declaración canónica
[1, Teorema 1.7]	`RefinedAbsorption.Faithful2024.theorem_1_7`
[1, Teorema 1.11]	`RefinedAbsorption.Faithful2024.theorem_1_11`
[1, Teorema 6.1]	`RefinedAbsorption.Original2024.theorem_6_1`
[1, Teorema 2.8]	`RefinedAbsorption.Pending.theorem_2_8`
[1, Teorema 6.2], cláusula ambiente explícita	`RefinedAbsorption.Faithful2024.theorem_6_2_ambient`
[1, Teorema 6.3]	`RefinedAbsorption.Pending.theorem_6_3`

Los dos primeros resultados conservan conjuntos explícitos de vértices del grafo ambiente y la distinción entre uso acotado de aristas, tamaño acotado de miembros y grado de refinamiento de una familia. El teorema de reserva usa copias etiquetadas de aristas, cotas de grados en las raíces designadas y la intersección ambiente indicada. No sustituye ese contrato por casi regularidad global ni por una cota del número de aristas.

El resultado anterior `RefinedAbsorption.Pending.theorem_6_2` sigue disponible. Su predicado de absorbedor usa el soporte de aristas del grafo antiguo. El resultado añadido también prohíbe una arista del absorbedor completamente contenida en el conjunto antiguo entero. Incluye etiquetas aisladas y permite aristas mixtas antiguas/nuevas. El espacio de nombres heredado `Pending` no significa que estos resultados demostrados sigan abiertos.

2.2. Survey

Fuente o precisión	Declaración canónica
[2, Teorema 1.10]	`RefinedAbsorption.Final.theorem_1_10`
[2, Conjetura 1.1], índice fijo al menos uno	`RefinedAbsorption.existenceConjecture_proved`
Especialización de existencia de Steiner	`RefinedAbsorption.Final.existenceConjecture_steiner`
Negación de la conclusión literal de encaje global	`RefinedAbsorption.FaithfulSurvey2025.not_generalEmbeddingLemmaLiteral`
Reemplazo por piezas demostrado	`RefinedAbsorption.FaithfulSurvey2025.generalEmbedding_piecewise`

Las dos últimas filas son afirmaciones distintas. El predicado global usa los vértices antiguos que aparecen en las piezas indexadas. El testigo con emparejamiento perfecto cubre todos los vértices antiguos, por lo que también refuta la conclusión impresa sobre el conjunto antiguo completo. No se infiere de ese testigo una identidad definicional general entre todas las convenciones sobre vértices no utilizados.

Se siguió la referencia de prueba de la survey [4, Apéndice B]. Su demostración existe; también escribe una inyección global. El teorema por piezas permite aplicaciones inyectivas separadas, con las condiciones sobre el grafo ambiente e imágenes distintas de aristas nuevas. No debe presentarse como una demostración de la afirmación global.

2.3. Research note

Ubicación en la nota	Declaración canónica
Teorema 5.1	`RefinedAbsorption.Extensions.steiner_completion_avoiding`
Corolario 5.2(i)	`RefinedAbsorption.Extensions.steiner_intersection_prescribed`
Corolario 5.2(ii)	`RefinedAbsorption.Extensions.partialSteiner_intersection_embedding`
Proposición 5.3	`RefinedAbsorption.Extensions.steiner_intersection_spectrum_poly`
Teorema 6.2, todo sistema existente	`RefinedAbsorption.Assembly.steiner_quantitatively_almost_resolvable`
Teorema 6.2, existencia	`RefinedAbsorption.Assembly.exists_quantitatively_almost_resolvable_steiner`
Teorema 6.3	`RefinedAbsorption.Assembly.nash_williams_scheduled_decomposition`

Los resultados de intersección están en `RefinedAbsorption/Extensions/Intersection.lean`. Usan el teorema de descomposición compatible, la completación dispersa usual y lemas de packings finitos. Los tres resultados cuantitativos cerrados están en `RefinedAbsorption/Assembly.lean`. Sus predecesores condicionales se conservan sin cambios.

3. Cuantificadores y representación

Para completación e intersecciones dispersas prescritas se fijan primero $q > r \geq 1$. La constante positiva de dispersión y el umbral de orden se eligen antes del orden ambiente y de los sistemas parciales. La dispersión se refiere a $\Delta_{r-1}(H(P))$, el r-grafo cubierto. El sistema parcial prohibido no tiene que ser disperso. La garantía adicional de cruce se restringe a bloques prohibidos cuyas r-aristas evitan $H(P)$.

Para existencia con índice arbitrario se fijan $q > r \geq 2$ y $\lambda \geq 1$ antes de elegir el umbral de orden. El umbral puede depender de los tres parámetros. La condición de admisibilidad usa ese índice; no se afirma un umbral uniforme para todos los índices.

Para el encaje como intersección exacta, un sistema parcial finito dado determina un umbral de orden. Todo orden admisible mayor admite una inyección y dos sistemas cuya familia común de bloques es precisamente la imagen de la familia dada. La inyección se elige junto con los sistemas; el resultado no cuantifica sobre toda inyección prescrita de antemano. El resultado de espectro elige ambos sistemas y da un intervalo inicial $0 \leq k \leq cn^r$.

El resultado universal de coloreo de Steiner fija $q > r \geq 2$ y después elige γ, n_0 antes de n y del sistema de Steiner. La variante de existencia añade admisibilidad. Lean escribe el cociente de grado de puntos como división natural antes de convertirlo a reales:

$$R = \frac{\binom{n-1}{r-1}}{\binom{q-1}{r-1}}.$$

La prueba de conteo establece divisibilidad exacta para un sistema de Steiner, por lo que éste es el grado ordinario de puntos, y no una aproximación por piso. La aplicación formal de colores

está definida sobre conjuntos finitos de vértices y se restringe en los bloques del sistema. Su número de etiquetas satisface

$$R \le m \le R + R^{1-\gamma}.$$

Se permiten etiquetas no usadas. Por eso la nota afirma una cota superior para el número de clases paralelas parciales y justifica la cota inferior por separado.

Para la aplicación a grafos existen constantes universales γ, n_0 antes del grafo finito. Las hipótesis son divisibilidad en triángulos, orden suficientemente grande y la desigualdad exacta

$$3n \le 4\delta(G).$$

La salida es una descomposición efectiva en triángulos y un coloreo propio de su hipergrafo de triángulos, con

$$\Delta(G)/2 \le m \le \Delta(G)/2 + (\Delta(G)/2)^{1-\gamma}.$$

No aparece una hipótesis de casi regularidad del grafo, un teorema externo como premisa ni una conclusión de factores que abarquen todos los vértices. Ambas aplicaciones cuantitativas tienen $0 < \gamma < 1$.

Los tipos finitos y las instancias de igualdad decidible son parámetros de representación. Los contratos seleccionados usan tipos finitos ordinarios en el universo cero. Las familias de bloques son conjuntos finitos, por lo que el resultado de existencia para índice fijo no permite repetir bloques. El nombre `steiner_orthogonal_mate` en otra parte de la biblioteca significa intersecciones cruzadas acotadas y ausencia de bloques comunes; no afirma ortogonalidad clásica de Steiner.

4. Ensamblaje de pruebas externas

La revisión fijada de Pippenger–Spencer es

`b19273396d369332fb489fe07e61e27c9e7f9233`

en `jtraverso/pippenger-spencer`. El fragmento instalado contiene `Basic.lean`, `Colour Hypergraph.lean` y `Polynomial.lean` bajo `RefinedAbsorption/Upstream/Pippenger Spencer/`. Sólo cambiaron las rutas de imports y los finales de línea. Se verificó la identidad de diecinueve fuentes compartidas de emparejamiento tras sustituir el prefijo de imports, y se reutilizan desde la dependencia Nash–Williams. Los hashes originales e instalados, incluida la normalización CRLF/LF, están registrados en `provenance/assembly-sources.json`.

El resultado de coloreo polinómico es `Pseudorandom.chromatic_index_polynomial`. Se demuestra a través del resultado de Ehard–Glock–Joos [6], usando un hipergrafo auxiliar de colores y una completación. Su contrato exacto incluye grado máximo, cogrados polinómicos y una cota exponencial del número de aristas. No tiene condición inferior de grados. Esto no afirma que [5] imprima esa formulación cuantitativa exacta.

La dependencia Nash–Williams es `jtraverso/nash-williams-lean` en

`2603197abdbf0a600ec663f6f046f70fbd209d4a`.

El resultado efectivamente utilizado es `NashWilliams18.PaperI.nash_williams_via_paper II`, importado desde `RequestProject.NW18.PaperIIAssembly`. El módulo local `Refined Absorption.ExternalProofs` proporciona ambos contratos mediante aplicación directa de esos resultados demostrados. Por tanto, las tres declaraciones finales de `RefinedAbsorption. Assembly` no tienen ninguna de las dos hipótesis de teorema externo.

La raíz local de módulos se trasladó de `RequestProject` a `RefinedAbsorption` para evitar una colisión con las rutas de la dependencia. Se conservaron los espacios de nombres de las

declaraciones matemáticas. El repositorio incluye la licencia del fragmento de fuentes y los registros de correspondencia.

5. Compilación, axiomas y conservación

El commit de fuente formal es

`5428ee9b772e67dd7ec6da443199b36a003b72d4`.

Lo identifica la etiqueta local de fuente formal `formal-v1.0`. El toolchain es `leanprover/lean4:v4.35.0-rc3`; Mathlib está fijado en

`c55e6e786f49471c72fbddbec5415808896aec1e`.

El manifiesto fija diez dependencias Git, incluida Nash–Williams. Se comprobaron las revisiones efectivas de sus checkouts y la ausencia de cambios en archivos versionados. Los targets por defecto de Lake son `RefinedAbsorption`, `checks` y `Palomar`. La raíz agregada importa los 300 módulos locales de biblioteca. Diez módulos de comprobación y seis interfaces independientes son targets explícitos adicionales, lo que da 316 fuentes locales.

La campaña comenzó sin directorio de salida del proyecto. Se reutilizaron cachés de dependencias fijadas; no se afirma una recompilación desde cero de Mathlib y de todo el desarrollo dependiente. Tras 129 módulos completos se reanudó el planificador acotado mediante cadenas de anchura uno, verificando hashes de fuentes y trazas genuinas de Lake. La continuación incluyó dos módulos adicionales de fidelidad y completó los 310 targets de biblioteca y comprobación, seguidos del build por defecto real. Después se registraron las seis interfaces de Palomar y se compilaron secuencialmente, seguidas de otro build por defecto exitoso.

Un intento de la raíz agregada falló porque la documentación del módulo precedía a los imports. Mover ese comentario después de ellos corrigió el error de empaquetado. Un intento de reanudación también detectó un problema de decodificación de texto en Windows; se corrigió el helper para leer las trazas explícitamente como UTF-8. No cambió ninguna declaración matemática conservada. Los logs fallidos y la historia de la campaña corregida siguen en `publication/assembly/`.

Una vez confirmados en Git los inputs, el validador del freeze comparó cada fuente Lean y archivo de configuración con los bytes del commit, ejecutó el control de vigencia rehash/no-build de Lake, comprobó trazas genuinas y registró hashes de fuentes y artefactos. Es la continuación de una campaña limpia del proyecto, no una afirmación de que cada módulo se compiló exactamente una vez.

Módulo de comprobación	Consultas
`AxiomCheck`	290
`ExtensionsAxioms`	14
`FaithfulAxioms`	13
`IntersectionAxioms`	11
`Original2024Axioms`	14
`PendingAxioms`	2
`PendingEndpoints`	3
`QuantitativeAxioms`	7
`AssemblyAxioms`	5
`SourceFidelityAxioms`	1
Total	360

Son consultas, sin afirmar que haya 360 nombres de teoremas distintos. Las salidas contienen sólo `propext`, `Classical.choice` y `Quot.sound`, o ningún axioma. El desarrollo de pruebas

conservado no tiene huecos pendientes ni axiomas del proyecto. Los Challenges contienen deliberadamente dieciocho huecos de enunciados; sus Solutions no los importan y proporcionan las pruebas reales. Sería incorrecto decir «sin axiomas».

Se conservan las 300 fuentes Lean originales retornadas. La comparación de migración no informa diferencias textuales en encabezados de declaraciones ni cambios en 821 intervalos de declaraciones no demostrativas normalizados respecto de comentarios y espacios. El traslado posterior de imports conserva las 301 fuentes migradas salvo la sustitución documentada de prefijos. Son controles textuales, no un teorema de equivalencia de kernel entre versiones. El registro anterior de migración mediante Lean directo, con 301 fuentes y 354 consultas, permanece como evidencia histórica y no reemplaza el certificado final de Lake.

6. Archivo y reproducción

El archivo formal inmutable es

`publication/formal-freeze/formal-v1.0-5428ee9b772e.tar.gz`,

con SHA-256

`e9bb21b17a110a392a7751afefb9a2e9d6bf218c7e6dac788006234f3af09c05`.

Contiene el commit de fuente y la evidencia completa de compilación. El recibo externo `FREEZE.json` y el sidecar SHA identifican ese archivo. El registro de validación `publication/assembly/VALIDATION.json` tiene SHA-256

`45321434a170482d8473182ecc8c1fd60b66a84bb9480ef7f6c26f9f2b130157`.

Desde el checkout completo de publicación, `python tools/verify_formal_freeze.py` comprueba la correspondencia de fuentes actuales y archivadas, la integridad del archivo, el archivo de fuentes originales y los logs de axiomas registrados. No ejecuta Lean ni reproduce independientemente las pruebas. El archivo formal no puede contener su propio recibo con hash; para ese verificador de conveniencia se usa el checkout completo de publicación.

Para reconstruir las pruebas desde cero, se usa un checkout separado del commit de fuente formal o se extrae el archivo formal, se instala el toolchain de Lean fijado y se ejecuta:

```
lake exe cache get
lake build
```

El manifiesto público usa revisiones Git inmutables. Las rutas locales de overrides usadas en la campaña registrada de Windows están ignoradas por Git y no se necesitan en esta configuración portable. Los hashes de artefactos compilados describen la plataforma registrada; una reconstrucción en otra plataforma no tiene por qué reproducir esos bytes binarios.

Los manuscritos se mantienen en Markdown inglés y español, y después se convierten con la plantilla de la serie Nash–Williams del autor a TeX final y PDF. La inspección de páginas renderizadas, la paridad bilingüe y los hashes documentales están separados del freeze de fuentes formales. Commits posteriores de documentos y metadatos pueden conservar los bytes congelados de Lean y configuración; los commits exactos de publicación y registro se anotan en sus propios recibos.

7. Límites de comparación y revisión

Las configuraciones de comparación son `Palomar/Article2024.comparator.json`, `Palomar/Survey2025.comparator.json` y `Palomar/ResearchNote.comparator.json`. Cada una tiene un Challenge independiente que importa sólo Mathlib y un Solution que importa las pruebas pertinentes. Las definiciones de los Challenges tienen cuerpos especificados; sólo los enunciados de teoremas tienen huecos deliberados.

La compilación local y el informe de axiomas establecen la evidencia descrita arriba. El preflight completo de Palomar, la comparación Challenge/Solution, la comprobación independiente de kernel, la revisión editorial y el registro son etapas adicionales con recibos separados. Este complemento no convierte un build local en uno de esos certificados.

La revisión interna la realiza Codex bajo la dirección del autor y se registra como autoevaluación. Aristotle contribuyó al desarrollo de pruebas y a la formalización; Codex contribuyó a la migración, el ensamblaje, la interfaz ambiente añadida, la exposición y la preparación de publicación. El autor asume la responsabilidad del trabajo. No se afirma una revisión independiente por pares humanos.

Referencias

[1] M. Delcourt and L. Postle, *Refined Absorption: A New Proof of the Existence Conjecture*, arXiv:2402.17855v1 (2024). https://doi.org/10.48550/arXiv.2402.17855.

[2] L. Postle, *Refined Absorption: A New Proof of the Existence Conjecture and Its Applications to Extremal and Probabilistic Design Theory*, Proceedings of the International Congress of Mathematicians 2026, Volume 6, 212–231. https://doi.org/10.1137/25M1805977. Base de enunciados: arXiv:2510.19978v1 (2025), https://doi.org/10.48550/arXiv.2510.19978.

[3] M. Delcourt and L. Postle, *Finding an Almost Perfect Matching in a Hypergraph Avoiding Forbidden Submatchings*, arXiv:2204.08981 (2022). https://doi.org/10.48550/arXiv.2204.08981.

[4] M. Delcourt, C. Henderson, T. Lesgourgues and L. Postle, *Erdős meets Nash-Williams*, arXiv:2507.23624v1 (2025), Appendix B. https://doi.org/10.48550/arXiv.2507.23624.

[5] N. Pippenger and J. Spencer, *Asymptotic behavior of the chromatic index for hypergraphs*, Journal of Combinatorial Theory, Series A 51 (1989), 24–42. https://doi.org/10.1016/0097-3165(89)90074-5.

[6] S. Ehard, S. Glock and F. Joos, *Pseudorandom hypergraph matchings*, Combinatorics, Probability and Computing 29(6) (2020), 868–885. https://doi.org/10.1017/S0963548320000280.

[7] M. Delcourt and L. Postle, *A Proof of Nash-Williams' Conjecture*, arXiv:2606.11178v1 (2026). https://doi.org/10.48550/arXiv.2606.11178.

[8] J. P. Traverso Gianini, *Triangle decompositions I: Formalizing the Nash–Williams theorem in Lean*, preprint 1.0 (2026). https://doi.org/10.5281/zenodo.23428173.

[9] J. P. Traverso Gianini, *Triangle decompositions II: Robust absorption for typed triangles*, preprint 1.0 (2026). https://doi.org/10.5281/zenodo.23502224.

[10] C. C. Lindner and A. Rosa, *Steiner triple systems having a prescribed number of triples in common*, Canadian Journal of Mathematics 27(5) (1975), 1166–1175. https://doi.org/10.4153/CJM-1975-122-4.

[11] C. C. Lindner and A. Rosa, *Steiner triple systems having a prescribed number of triples in common (corrigendum)*, Canadian Journal of Mathematics 30(4) (1978), 896. https://doi.org/10.4153/CJM-1978-077-9.

[12] L. de Moura and S. Ullrich, *The Lean 4 Theorem Prover and Programming Language*, Automated Deduction — CADE 28 (2021), 625–635. https://doi.org/10.1007/978-3-030-79876-5_37.

[13] The mathlib Community, *The Lean Mathematical Library*, Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (2020), 367–381. https://doi.org/10.1145/3372885.3373824.